

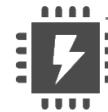
# K-means Clustering을 위한 FPU\_IP 설계 프로젝트 안내

이우주교수님 연구실

박진아



중앙대학교  
CHUNG-ANG UNIVERSITY



Low Power SoC Lab.

중앙대학교 저전력 SoC 연구실

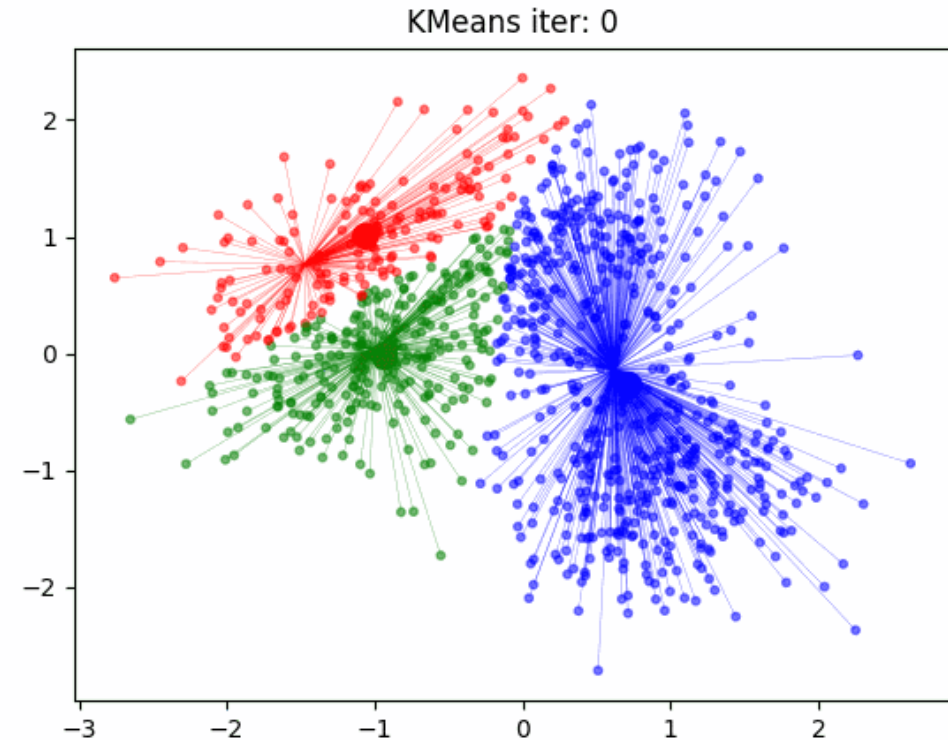
## [Contents]

- 01. K-means clustering**
- 02. FPU (Floating Point Unit)
- 03. 프로젝트 안내
- 04. 첨부 파일 (kmeans\_fpu.zip)

# K-means clustering

“K-means Clustering 특화 경량 RISC-V 프로세서”  
제22회 대한민국 반도체설계대전 산업통상자원부 장관상 수상

- ▶ 정답 레이블을 사용하지 않는  
비지도 학습(Unsupervised Learning)
- ▶ 데이터를 주어진 **K 개의 클러스터로 군집화**하는  
간단하고 효율적인 방법



# K-means clustering : Original code

```
C k_means_oled.c •
kmeans > src > C k_means_oled.c > ...
1
2 /* K-means Algorithm, kjbyun, 2020-08-07 */
3
4 #include "platform_info.h"
5 #include "ervp_printf.h"
6 #include "ervp_core_id.h"
7 #include "ervp_delay.h"
8 #include "ervp_profiling.h"
9 #include "oled_rgb.h"
10
11 #define OLED_W 96
12 #define OLED_H 64
13
14 #define k 3 // # of cluster
15 #define num_data 5 // # of input data
16 #define sqr(x) ((x)*(x))
17 #define ABS(x) ((x) < 0 ? -(x) : (x))
18
19 #include "data_150.h" // 2 dimension input data : data[num_data][2]
20 #define data data_150
21
22 #define use_oled 0
23 #define full_printf 0
24
25 int main()
26 {
27     if(EXCLUSIVE_ID==0)
28     {
29         float means[k][2];
30         int group[num_data];
31         int iteration = 0;
32         int clust_data[num_data][2];
33         int ii = 0, g[k];
34
35         // set initial means //
36         for(int i = 0; i < k; i++)
37             for(int j = 0; j < 2; j++)
38                 means[i][j] = (float)data[i][j];
39
40
41         if(full_printf) {
42             printf("Start computing K-means algorithm : initial means\n");
43
44             for (int i = 0; i < k; i++)
45                 printf("means[%d] = (%.2f, %.2f)\n", i, means[i][0], means[i][1]);
46         }
47
48         if(use_oled) {
49             // display input data and initial means //
50             printf("Draw the input data and initial means for K-means clustering\n");
51             oled_rgb_start();
52             oled_rgb_clear();
53
54             // Draw axis
55             for(int i = 0; i < OLEDRGB_HEIGHT; i++)
56                 oled_rgb_draw_pixel(OLEDRGB_WIDTH+1, i, RGB(0xFF,0xFF,0xFF));
57
58             for(int i = 1; i < OLEDRGB_WIDTH; i++)
59                 oled_rgb_draw_pixel(i, OLEDRGB_HEIGHT-1, RGB(0xFF,0xFF,0xFF));
60
61             delay_ms(1000);
62         }
63     }
64 }
```

```
81
82 // K-means clustering //
83 for(;;)
84 {
85     int flag = 0;
86
87     for(int i = 0; i < num_data; i++)
88     {
89         float min_dis = 9999;
90         int min = -1;
91
92         // calculating distances //
93         for(int j = 0; j < k; j++)
94         {
95             float dis = 0;
96
97             for(int l = 0; l < 2; l++)
98                 dis += sqr((float)data[i][l] - means[j][l]);
99
100             if(dis < min_dis) {
101                 min_dis = dis;
102                 min = j;
103             }
104
105             // set groups //
106             group[i] = min;
107         }
108
109         // calculating new means for each cluster //
110         float temp[k][2];
111         int count[k];
112
113         for(int i = 0; i < k; i++)
114         {
115             count[i] = 0;
116             for(int j = 0; j < 2; j++)
117                 temp[i][j] = 0.f;
118         }
119
120         // calculating new means //
121         for(int i = 0; i < num_data; i++)
122         {
123             count[group[i]]++;
124             for(int j = 0; j < 2; j++)
125                 temp[group[i]][j] += (float)data[i][j];
126         }
127
128         for(int i = 0; i < k; i++)
129         {
130             for(int j = 0; j < 2; j++)
131             {
132                 temp[i][j] /= count[i];
133
134                 if(ABS(temp[i][j] - means[i][j]) > 0.0001)
135                 {
136                     flag++;
137                     means[i][j] = temp[i][j];
138                 }
139             }
140
141             iteration++;
142         }
143     }
144 }
```

```
146
147 // print the result of clustering : clustering point of view
148
149 ii = 0;
150 for(int i = 0; i < k; i++)
151 {
152     g[i] = 0;
153     for(int j = 0; j < num_data; j++)
154     {
155         if (group[j] == i)
156         {
157             clust_data[ii][0] = data[j][0];
158             clust_data[ii][1] = data[j][1];
159             g[i] += 1;
160             ii++;
161         }
162     }
163
164     ii = 0;
165     for(int i = 0; i < k; i++)
166     {
167         // printf("# of group %d data : %d\n", i, g[i]);
168         for(int j = 0; j < g[i]; j++)
169         {
170             // printf("%2d %2d\n", clust_data[ii][0], clust_data[ii][1]);
171             ii++;
172         }
173     }
174
175     if(full_printf) {
176         for(int i = 0; i < k; i++)
177             printf("means[%d] = (%.2f, %.2f)\n", i, means[i][0], means[i][1]);
178     }
179
180     if(use_oled) {
181         // display result of clustering for each iteration //
182         oled_rgb_clear();
183         printf("Draw the result of clustering for iteration %d\n", iteration);
184
185         // Draw axis
186         for(int i = 0; i < OLEDRGB_HEIGHT; i++)
187             oled_rgb_draw_pixel(OLEDRGB_WIDTH+1, i, RGB(0xFF,0xFF,0xFF));
188
189         for(int i = 1; i < OLEDRGB_WIDTH; i++)
190             oled_rgb_draw_pixel(i, OLEDRGB_HEIGHT-1, RGB(0xFF,0xFF,0xFF));
191
192         delay_ms(1000);
193
194         // Draw clustered data for each iteration
195         int i, j = 0;
196
197         for(i = 0; i < g[0]; i++)
198             oled_rgb_draw_pixel(clust_data[i][0], OLEDRGB_HEIGHT-clust_data[i][1], RGB(0xFF, 0, 0));
199
200         for(j = 1; j < g[0]+g[1]; j++)
201             oled_rgb_draw_pixel(clust_data[j][0], OLEDRGB_HEIGHT-clust_data[j][1], RGB(0, 0x2, 0));
202
203         for(i = j; i < g[0]+g[1]+g[2]; i++)
204             oled_rgb_draw_pixel(clust_data[i][0], OLEDRGB_HEIGHT-clust_data[i][1], RGB(0, 0, 0xFF));
205
206         delay_ms(1000);
207     }
208 }
```

# K-means clustering : FPU (Floating Point Unit)



```
COM9 - PuTTY
[EMU@FPGA]
Start computing K-means algorithm : initial means
means[0] = (77.00, 20.00)
means[1] = (48.00, 39.00)
means[2] = (52.00, 49.00)

iteration = 1
means[0] = (77.00, 18.23)
means[1] = (22.44, 39.00)
means[2] = (54.58, 49.85)

iteration = 2
means[0] = (77.00, 18.23)
means[1] = (19.72, 39.00)
means[2] = (54.58, 49.85)

iteration = 3
means[0] = (77.00, 18.23)
means[1] = (19.72, 39.00)
means[2] = (54.58, 49.85)

End of K-means clustering! iteration : 3
# of group 0 data : 35
# of group 1 data : 29
# of group 2 data : 36

[section] main
total tick 179985<<0
num_called 1
total time (ms) 179.985001
average time (ms) 179.99
```

- ▷ K-means clustering : iteration마다 새로운 군집중심값(mean)도출.
  - 실수연산!
- ▷ 이번 프로젝트에서 사용하는 코어는 경량 정수형 코어인 ORCA코어..
  - ▶ 실수연산을 수행하는 FPU(Floating Point Unit)이 없기 때문에 실수연산 속도가 매우 느림.
- FPU모듈을 IP에 탑재하여 실수연산속도를 높인다!

## [Contents]

01. K-means clustering
- 02. FPU (Floating Point Unit)**
03. 프로젝트 안내
04. 첨부 파일 (kmeans\_fpu.zip)

# FPU (Floating Point Unit)

## ▷ FPU? >> 부동소수점 장치(floating point unit)

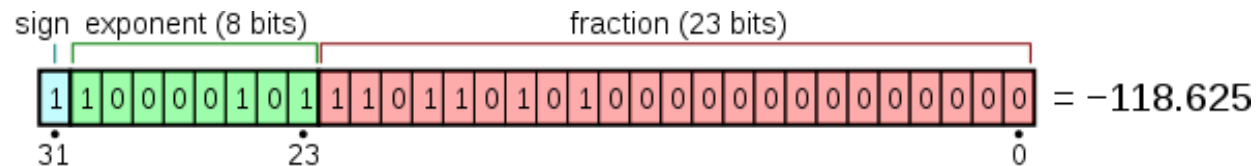
: 부동소수점 연산을 효율적으로 처리하기 위한 하드웨어 논리회로 모듈

## ▷ 부동소수점(floating point)?

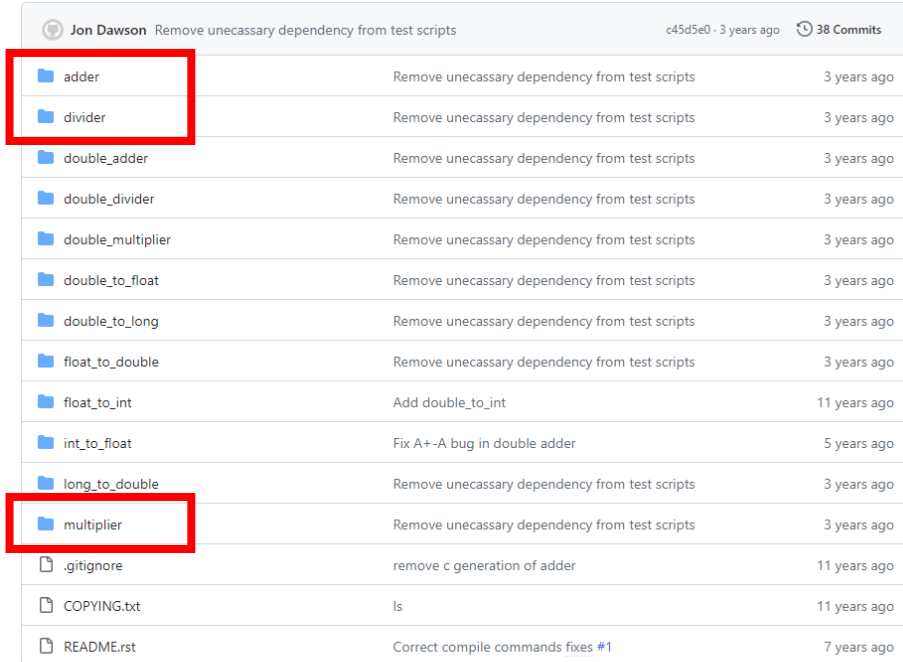
: 실수를 컴퓨터상에서 근사하여 표현할 때 소수점의 위치를 고정하지 않고 그 위치를 나타내는 수를 따로 적는 것으로, 유효숫자를 나타내는 가수(假數)와 소수점의 위치를 풀이하는 지수(指數)로 나누어 표현한다.

: 고정 소수점 방식보다 넓은 범위의 수를 나타낼 수 있음.

: 고정 소수점 방식보다 **연산 속도가 느리기** 때문에  
별도의 전용 연산 장치(ex, FPU)를 두는 경우가 많음.



# FPU (Floating Point Unit) : Verilog code (git)



|                                                                                            |                                                |              |
|--------------------------------------------------------------------------------------------|------------------------------------------------|--------------|
| Jon Dawson Remove unecassary dependency from test scripts c45d5e0 · 3 years ago 38 Commits |                                                |              |
| adder                                                                                      | Remove unecassary dependency from test scripts | 3 years ago  |
| divider                                                                                    | Remove unecassary dependency from test scripts | 3 years ago  |
| double_adder                                                                               | Remove unecassary dependency from test scripts | 3 years ago  |
| double_divider                                                                             | Remove unecassary dependency from test scripts | 3 years ago  |
| double_multiplier                                                                          | Remove unecassary dependency from test scripts | 3 years ago  |
| double_to_float                                                                            | Remove unecassary dependency from test scripts | 3 years ago  |
| double_to_long                                                                             | Remove unecassary dependency from test scripts | 3 years ago  |
| float_to_double                                                                            | Remove unecassary dependency from test scripts | 3 years ago  |
| float_to_int                                                                               | Add double_to_int                              | 11 years ago |
| int_to_float                                                                               | Fix A+-A bug in double adder                   | 5 years ago  |
| long_to_double                                                                             | Remove unecassary dependency from test scripts | 3 years ago  |
| multiplier                                                                                 | Remove unecassary dependency from test scripts | 3 years ago  |
| .gitignore                                                                                 | remove c generation of adder                   | 11 years ago |
| COPYING.txt                                                                                | Is                                             | 11 years ago |
| README.rst                                                                                 | Correct compile commands fixes #1              | 7 years ago  |

README MIT license

## IEEE 754 floating point arithmetic

Synthesiseable IEEE 754 floating point library in Verilog.

- Provides Divider, Multiplier and Adder
- Provides float\_to\_int and int\_to\_float
- Supports Denormal Numbers
- Round-to-nearest (ties to even)
- Optimised for area
- Over 100,000,000 test vectors (for each function)

git 링크 : <https://github.com/dawsonjon/fpu/tree/master>  
**"Synthesiseable IEEE 754 floating point library in Verilog"**

Git의 adder, divider, multiplier 코드를  
프로젝트에서 사용할 수 있도록 RVX 환경에 맞게 수정하여  
압축 파일(kmeans\_fpu\_project.zip)에 포함시켜 두었습니다.

- **fpu\_adder.v, fpu\_divider.v, fpu\_multiplier.v**
- 반드시 파일로 첨부해드린 위 코드를 instance하여 사용하시길 바랍니다.
- **(절대 수정 X)(git 코드 직접 받아서 사용 X)**

## [Contents]

- 01. K-means clustering
- 02. FPU (Floating Point Unit)
- 03. 프로젝트 안내**
- 04. 첨부 파일 (kmeans\_fpu.zip)

# 프로젝트 안내

## Step 1. FPU모듈 동작 확인하기 (RTL Simulation)

- IEEE754 FPU verilog code 이해, testbench 작성, 시뮬레이션 돌려보기

## Step 2. FPU모듈 적용한 IP 설계 (RTL Simulation, FPGA Prototyping)

- FPU모듈을 사용하는 IP 설계, 프로세서에 연결, 사칙연산 test app 실행해보기

## Step 3. 실제 APP(K-means clustering)에 설계한 IP 적용하기 (RTL Simulation, FPGA Prototyping)

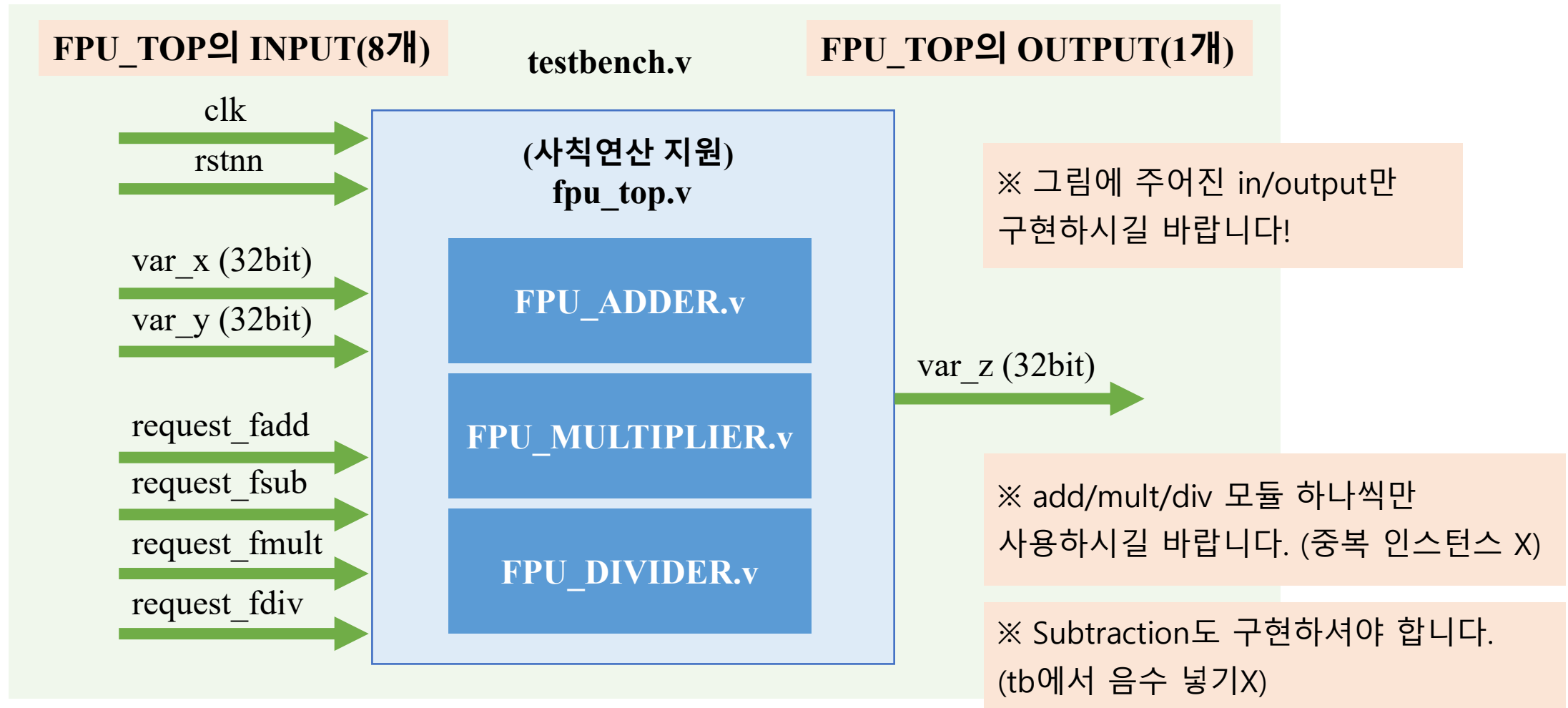
- K-means clustering app에 설계한 IP를 적용, 정상작동하는지 확인하기

# 프로젝트 안내 : (Step 1) FPU모듈 동작 확인하기

## Step 1. FPU모듈 동작 확인하기 (RTL Simulation)

1. IEEE754 FPU verilog code 이해
  2. Top Module(fpu\_top.v), testbench 작성
  3. Questa/Vivado로 RTL 시뮬레이션 진행
- ✓ kmeans\_fpu.zip/rtl : fpu\_adder.v, fpu\_multiplier.v, fpu\_divider.v (수정X)

# 프로젝트 안내 : (Step 1) FPU모듈 동작 확인하기



# 프로젝트 안내 : (Step 1) FPU모듈 동작 확인하기

```
fpu_adder.v fpu_divider.v fpu_multiplier.v TEST1_APB.v
1 //IEEE Floating Point Adder (Single Precision)
2 //Copyright (C) Jonathan P Dawson 2013
3 //2013-12-12
4
5 module fpu_adder(
6     input_a,
7     input_b,
8     input_a_stb,
9     input_b_stb,
10    output_z_ack,
11    clk,
12    rstnn,
13    output_z,
14    output_z_stb,
15    input_a_ack,
16    input_b_ack);
17
18    input    clk;
19    input    rstnn;
20
21    input    [31:0] input_a;
22    input    input_a_stb;
23    output   input_a_ack;
24
25    input    [31:0] input_b;
26    input    input_b_stb;
27    output   input_b_ack;
28
29    output   [31:0] output_z;
30    output   output_z_stb;
31    input    output_z_ack;
32
```

## ❖ 32bit in/output 데이터

**input\_a, input\_b** : 피연산자

**output\_z** : 결과값

## ❖ Handshake 신호

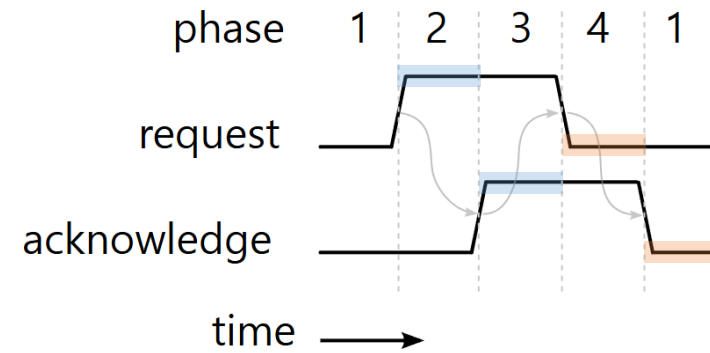
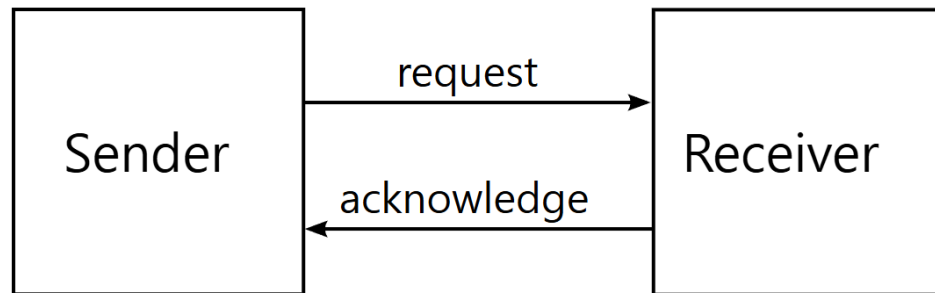
**XX\_stb** : 데이터 전송 전 발생 신호  
(데이터가 준비되었음)

**XX\_ack** : 데이터 수신 후 발생 신호  
(데이터를 성공적으로 수신하였음)

※ 모든 handshake 신호를 적절히 사용하시길 바랍니다!

# 프로젝트 안내 : (Step 1) FPU모듈 동작 확인하기

## ❖ Handshake 예시



# 프로젝트 안내 : (Step 1) FPU모듈 동작 확인하기 : FAQ

▷ (Step 1) FPU모듈 동작 확인하기 까지 했을 때 알 수 있는 것 :

1. FPU Adder/Multiplier/Divider의 사용법 확인
2. 설계한 FPU사칙연산 제어로직(피연산자  $x$ ,  $y$ , request 신호가 들어오면 사칙연산을 수행하고 연산결과값  $z$  를 만들어내는 로직)의 최소한의 동작 검증

▶ But, 아직 간단한 RTL simulation만 해본 것이므로,

- 1) 다른 app.을 돌릴 때에도 정상작동 할지
- 2) 이 코드가 synthesizable 한 코드인지
- 3) FPGA에 올렸을 때에도 정상작동 할지 아직 불확실한 상태...

➤ 그럼 이제 설계한 FPU사칙연산 로직(fpu\_top.v)을 사용해서 실제 IP를 설계해봅시다!

# 프로젝트 안내 : (Step 2) FPU모듈 적용한 IP 설계

## Step 2. FPU모듈 적용한 IP 설계 (RTL Simulation, FPGA Prototyping)

1. Step 1에서 설계한 fpu\_top.v를 IP\_TOP.v 에 인스턴스 (fpu\_top.v 코드 수정 가능)
  2. 프로세서에 연결
  3. fpu\_test.c 실행 (RTL Simulation, FPGA Prototyping)
- ✓ kmeans\_fpu.zip/rtl : **IP\_TOP.v (빈칸 채워서 완성)**
  - ✓ kmeans\_fpu.zip/app : **fpu\_test.c (수정X)**

# 프로젝트 안내 : (Step 2) FPU모듈 적용한 IP 설계

```
1 // IP_TOP.v
2 `include "ervp_global.vh"
3
4 module IP_TOP
5 (
6     clk,
7     rstnn,
8
9     rpsel,
10    rpenable,
11    rpaddr,
12    rpwrite,
13    rpwdata,
14    rprdata,
15    rpready,
16    rpslverr
17 );
18
19 // parameter input output */
20 ///////////////////////////////////////////////////
21
22 parameter BW_ADDR = 1;
23 parameter BW_DATA = 1;
24
25 input wire clk, rstnn;
26 input wire rpsel;
27 input wire rpenable;
28 input wire [BW_ADDR-1:0] rpaddr;
29 input wire rpwrite;
30 input wire [BW_DATA-1:0] rpwdata;
31 output reg [BW_DATA-1:0] rprdata;
32 output wire rpready;
33 output reg rpslverr;
34
35 ///////////////////////////////////////////////////
36 /* signals */
37 ///////////////////////////////////////////////////
38
39 genvar i;
40
41 wire [8-1:0] addr_offset = rpaddr;
42 reg is_valid_addr;
43
44 reg [32-1:0] var_x, var_y, var_z;
45 reg we_x, we_y, we_z;
46
47 reg request_fadd;
48 reg request_fsub;
49 reg request_fmuilt;
50 reg request_fdiv;
51
52 reg rpready_set;
53
54 // Please fill in the relevent code here.
55 */
```

```
57 ///////////////////////////////////////////////////
58 /* logics */
59 ///////////////////////////////////////////////////
60
61 always@(*)
62 begin
63     rprdata = 0;
64     is_valid_addr = 0;
65     we_x = 0;
66     we_y = 0;
67     we_z = 0;
68     request_fadd = 0;
69     request_fsub = 0;
70     request_fmuilt = 0;
71     request_fdiv = 0;
72
73     if(rpsel==1'b 1)
74     begin
75         if(rpenable==1'b 1)
76         begin
77             case(addr_offset)
78             8'h 0:
79             begin
80                 is_valid_addr = 1;
81                 rprdata = var_x;
82                 we_x = rpwrite;
83             end
84             8'h 4:
85             begin
86                 is_valid_addr = 1;
87                 rprdata = var_y;
88                 we_y = rpwrite;
89             end
90             8'h 8:
91             begin
92                 is_valid_addr = 1;
93                 rprdata = var_z;
94                 we_z = rpwrite;
95             end
96             8'h C:
97             begin
98                 is_valid_addr = 1;
99                 if(rpwrite==1'b 1)
100                 request_fadd = 1;
101                 else
102                 request_fsub = 1;
103             end
104             8'h 10:
105             begin
106                 is_valid_addr = 1;
107                 if(rpwrite==1'b 1)
108                 request_fmuilt = 1;
109                 else
110                 request_fdiv = 1;
111             end
112         end
113     end
114
115 // Please fill in the relevent code here.
116 */
117 end
```

IP\_TOP.v

```
119 always@(posedge clk, negedge rstnn)
120 begin
121     if(rstnn==0) var_x <= 0;
122     else if(we_x==1'b 1) var_x <= rpwdata;
123 end
124
125 always@(posedge clk, negedge rstnn)
126 begin
127     if(rstnn==0) var_y <= 0;
128     else if(we_y==1'b 1) var_y <= rpwdata;
129 end
130
131 always@(posedge clk, negedge rstnn)
132 begin
133     if(rstnn==0)
134     var_z <= 0;
135     else if(we_z==1'b 1)
136     var_z <= rpwdata;
137
138     /*
139     Please fill in the relevent code here.
140     */
141 end
142
143 /*
144 Please fill in the relevent code here.
145 (including module instantiations)
146 */
147
148 assign rpready = rpready_set;
149
150 always@(posedge clk, negedge rstnn)
151 begin
152     if(rstnn == 0) rpready_set <= 1;
153     /*
154     Please fill in the relevent code here.
155     */
156 end
157
158 always@(posedge clk, negedge rstnn)
159 begin
160     if(rstnn==0)
161     rpslverr <= 0;
162     else if(rpsel==1'b 1)
163     begin
164         if(rpenable==1'b 0)
165         rpslverr <= 0;
166         else
167         rpslverr <= ~is_valid_addr;
168     end
169 end
170
171 endmodule
172
173
174
```

# 프로젝트 안내 : (Step 2) FPU모듈 적용한 IP 설계

```
150 assign rpready = rpready_set;
151
152 always@(posedge clk, negedge rstnn)
153 begin
154     if(rstnn == 0) rpready_set <= 1;
155     /*
156     Please fill in the relevent code here.
157     */
158 end
```

## ❖ rpready 신호 주의사항

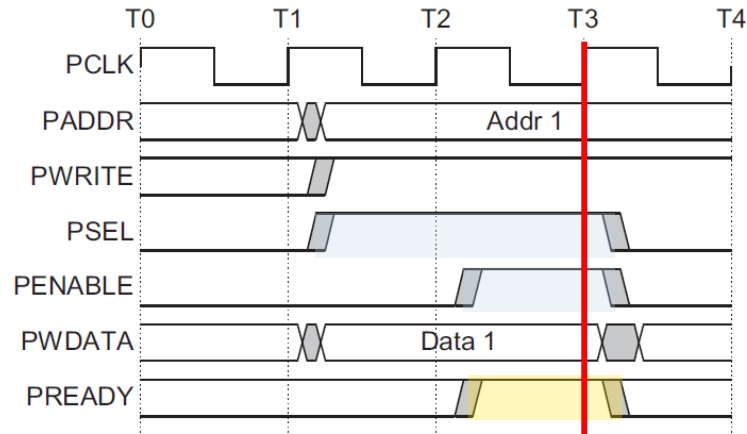
FPU모듈이 연산을 마치기에는 충분한 시간이 필요함!

- ▷ 따라서 올바른 output(연산결과값)을 얻기 위해서는 FPU모듈에 input을 넣은 다음, 충분히 기다려줘야 함.

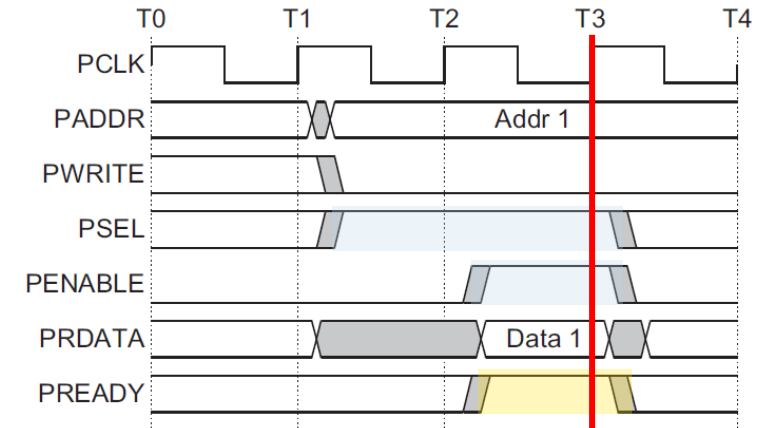
➤ **rpready**신호를 사용해서 타이밍 조절.

**AMBA APB의 pready**신호 참고하여  
빈 칸에 rpready\_set 로직 작성해주세요.  
(다음 장 waveform 참고)

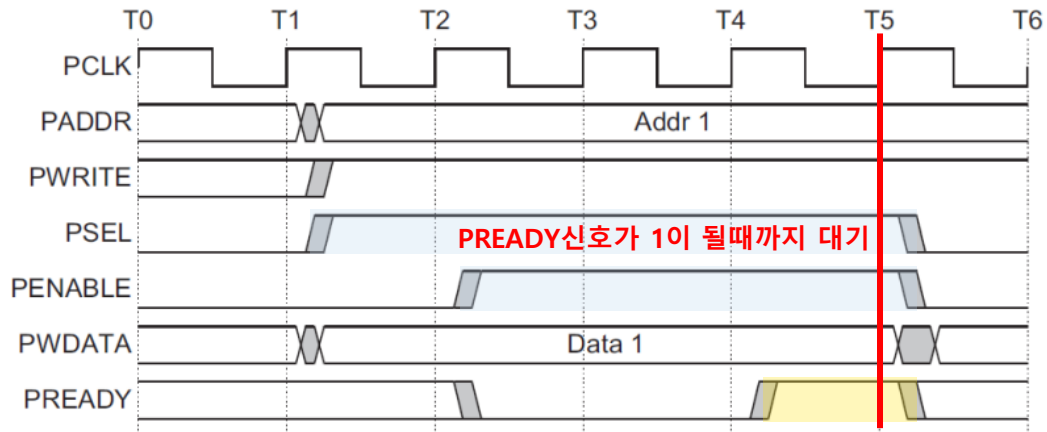
# 프로젝트 안내 : (Step 2) FPU모듈 적용한 IP 설계



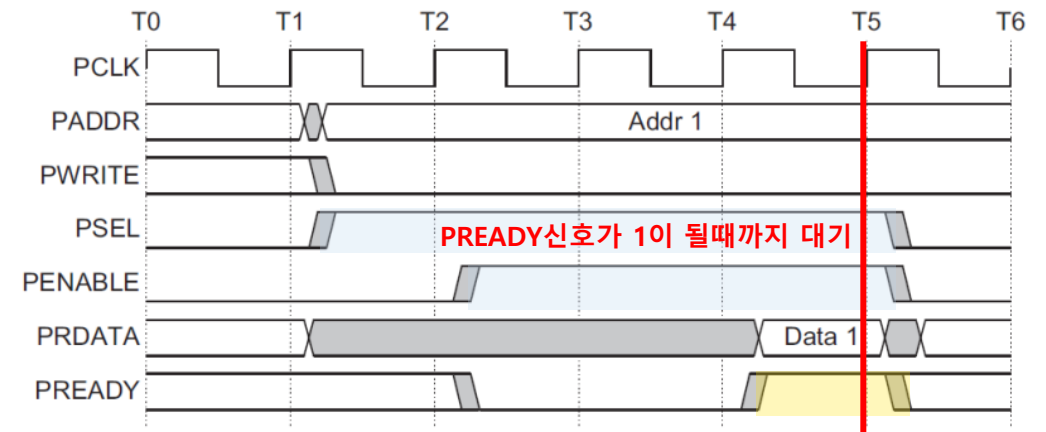
APB write with no wait states



APB read with no wait states



APB write with wait states



APB read with wait states

# 프로젝트 안내 : (Step 2) FPU모듈 적용한 IP 설계

```
fpu > src > C_main.c > ... fpu_test.c
1  #include "platform_info.h"
2  #include "ervp_printf.h"
3  #include "ervp_reg_util.h"
4
5  #define OFFSET_X 0x0
6  #define OFFSET_Y 0x4
7  #define OFFSET_Z 0x8
8  #define OFFSET_FADD 0xC
9  #define OFFSET_FSUB 0xC
10 #define OFFSET_FDIV 0x10
11 #define OFFSET_FMULT 0x10
12
13 #define REG32_fl(add) *((volatile float *) (add))
14
15 static inline unsigned int get_test1_addr(unsigned int offset)
16 {
17     return (I_TEST1_SLAVE_BASEADDR + offset);
18 }
19
20 void set_x(float value)
21 {
22     REG32_fl(get_test1_addr(OFFSET_X)) = value;
23 }
24
25 void set_y(float value)
26 {
27     REG32_fl(get_test1_addr(OFFSET_Y)) = value;
28 }
29
30 void set_z(float value)
31 {
32     REG32_fl(get_test1_addr(OFFSET_Z)) = value;
33 }
```

```
35 float get_x()
36 {
37     return REG32_fl(get_test1_addr(OFFSET_X));
38 }
39
40 float get_y()
41 {
42     return REG32_fl(get_test1_addr(OFFSET_Y));
43 }
44
45 float get_z()
46 {
47     return REG32_fl(get_test1_addr(OFFSET_Z));
48 }
49
50 void perform_fadd()
51 {
52     REG32(get_test1_addr(OFFSET_FADD)) = 0;
53 }
54
55 void perform_fsub()
56 {
57     int temp;
58     temp = REG32(get_test1_addr(OFFSET_FSUB));
59 }
60
61 void perform_fmult()
62 {
63     REG32(get_test1_addr(OFFSET_FMULT)) = 0;
64 }
65
66 void perform_fdiv()
67 {
68     int temp;
69     temp = REG32(get_test1_addr(OFFSET_FDIV));
70 }
```

```
73 int main()
74 {
75     set_x(14.53);
76     printf("\nx : %.2f", get_x());
77     set_y(87.91);
78     printf("\ny : %.2f", get_y());
79     set_z(670.72);
80     printf("\nz : %.2f", get_z());
81     perform_fadd();
82     printf("\nfadd : %.2f", get_z());
83     perform_fsub();
84     printf("\nfsub : %.2f", get_z());
85     perform_fmult();
86     printf("\nfmult : %.2f", get_z());
87     perform_fdiv();
88     printf("\nfdiv : %.2f", get_z());
89
90     return 0;
91 }
```

**fpu\_test.c (수정 X)**

# 프로젝트 안내 : (Step 2) FPU모듈 적용한 IP 설계 : FAQ

▷ (Step 2) FPU모듈 적용한 IP설계 까지 했을 때 알 수 있는 것 :

1. 설계한 IP가 APB통신을 통해 SoC에 잘 연결 되었는지 확인
2. 제안한 사칙연산 제어로직의 합성가능여부, FPGA프로토타이핑 시 정상동작 여부 확인

▶ But, 아직 간단한 어플리케이션만 돌려본 것이므로...

(여전히!) 다른 app.을 돌릴 때에도 정상작동할지 는 아직 불확실한 상태...  
(피연산자가 바뀌거나, 수행될 연산의 순서가 바뀌는 경우에 대한 검증 필요)

➤ 설계한 IP의 검증 & 활용을 위해 **K-means clustering app.**을 실행해봅시다!

# 프로젝트 안내 : (Step 3) 실제 APP에 IP 적용하기

## Step 3. 실제 APP(K-means clustering)에 설계한 IP 적용하기 (RTL Simulation, FPGA Prototyping)

1. k\_means\_clustering.c 안의 실수연산을 작성한 FPU연산 API로 대체
  2. k\_means\_clustering.c 실행 (RTL Simulation, FPGA Prototyping)
  3. FPU적용하지 않은 k\_means\_clustering.c 실행 결과와 비교하기
- ✓ kmeans\_fpu.zip/app : k\_means\_oled.c

# 프로젝트 안내 : (Step 3) 실제 APP에 IP 적용하기

## K-means코드 수정 예시

```
// calculating new means for each cluster //
float temp[k][2];

count[i] = 0;
for(int j = 0; j < 2; j++)
    temp[i][j] = 0.f;

// calculating new means //
for(int i = 0; i < num_data; i++)
{
    count[group[i]]++;
    for(int j = 0; j < 2; j++)
        temp[group[i]][j] += (float)data[i][j];

for(int i = 0; i < k; i++)
{
    for(int j = 0; j < 2; j++)
    {
        temp[i][j] /= count[i];

        if(ABS(temp[i][j] - means[i][j]) > 0.0001)
        {
            flag++;
            means[i][j] = temp[i][j];
        }
    }
}
```

perform\_fadd()

perform\_fdiv()

perform\_fsub()

설계한 IP를 검증하기 위해선  
당연히! 사칙연산을 모두 사용해봐야겠죠?

※ 사칙연산을 모두 사용해서  
K-means app.을 수정해주시길 바랍니다!

IP 적용 전/후 **군집화 결과**가 동일하다면?

⇒ 😎 IP가 잘 설계되었다는 뜻입니다.

⇒ 다른 app.에 적용했을 때도 (아마) 문제없이  
정상동작할 것입니다!

# 프로젝트 안내 : (Step 3) 실제 APP에 IP 적용하기

▷ 가속효과 확인하기 (num\_data 100, use\_oled 0, full\_printf 0)

parameter 설명은  
다음 슬라이드에 나옵니다!

```
[EMU@FPGA]
iteration = 3
means[0] = (70.43, 18.23) // 35
means[1] = (19.72, 35.90) // 29
means[2] = (54.08, 49.14) // 36

[section] main
total tick 197831<<0
num_called 1
total time (ms) 197.830994
average time (ms) 197.83

[section] K-means clustering μs
total tick 183673<<0
num_called 1
total time (ms) 183.673004
average time (ms) 183.67
```

```
profiling_start("K-means clustering");
```

```
/// k-means clustering ///
```

이 사이 시간 측정/출력

```
profiling_end("K-means clustering");
```

|                          | FPU_IP 적용 전 | FPU_IP 적용 후 |
|--------------------------|-------------|-------------|
| Performance (total tick) | ?           | ?           |

?? 배 가속

# K-means clustering 앱 내 parameter

```
11 #define OLED_W 96
12 #define OLED_H 64
13
14 #define k 3 // # of cluster
15 #define num_data 5 // # of input data
16 #define sqr(x) ((x)*(x))
17 #define ABS(x) (((x) < 0) ? -(x) : (x))
18
19 #include "data_150.h" // 2 dimension input data : data[num
20 #define data data_150
21
22 #define use_oled 0
23 #define full_printf 0
```

**num\_data : 0~150 설정 가능**

**RTL : 5 권장** (이 때 sim 5분 정도 소요됨)  
(data 수가 많아질수록 simulation 소요시간이 늘어납니다.)  
**FPGA : 100**

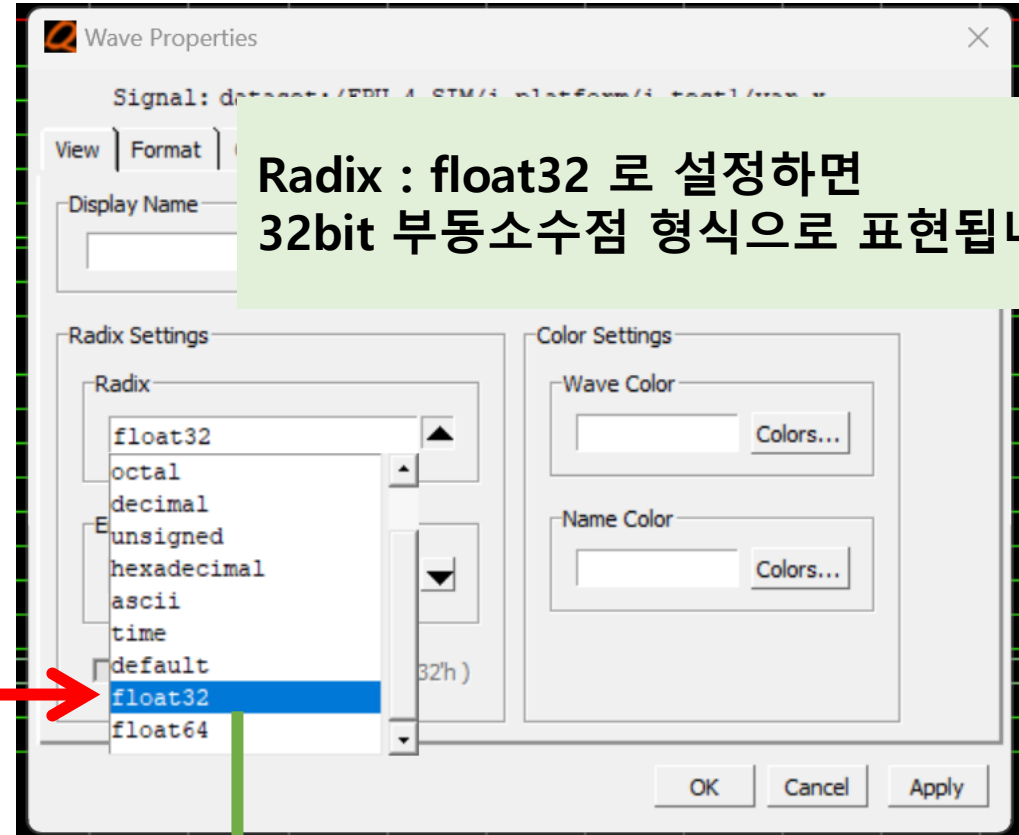
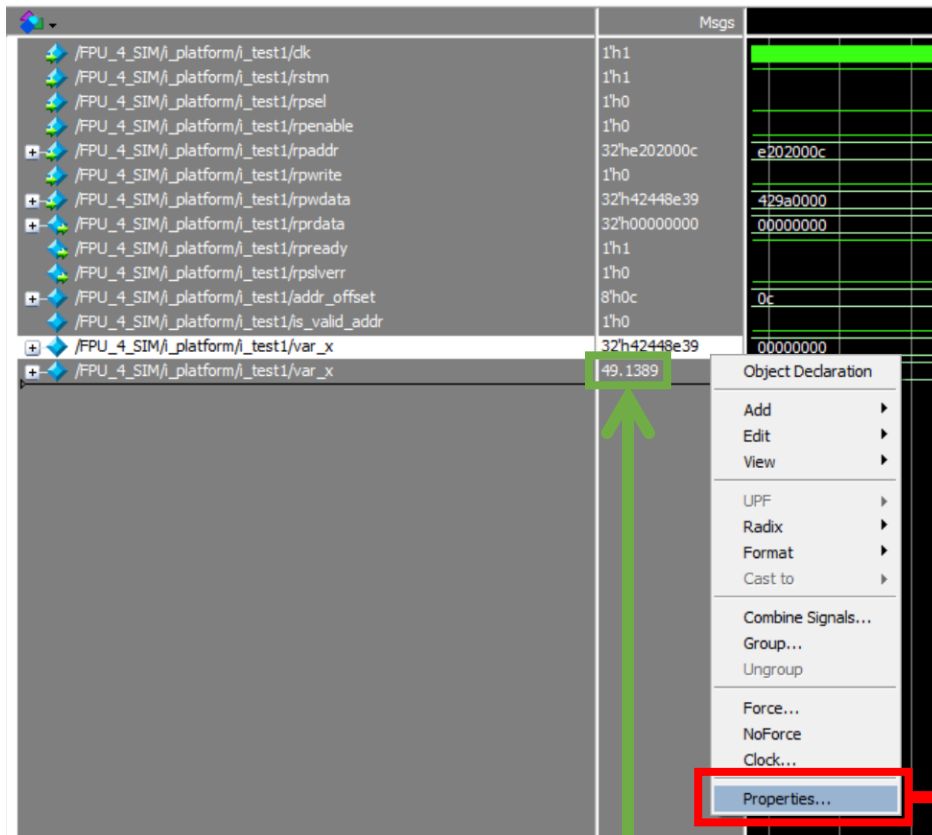
**use\_oled : 0 or 1** (oled 활성화 옵션)

**RTL : 0** (sim\_rtl에선 oled함수 실행 불가능.)  
**FPGA : 0 or 1**

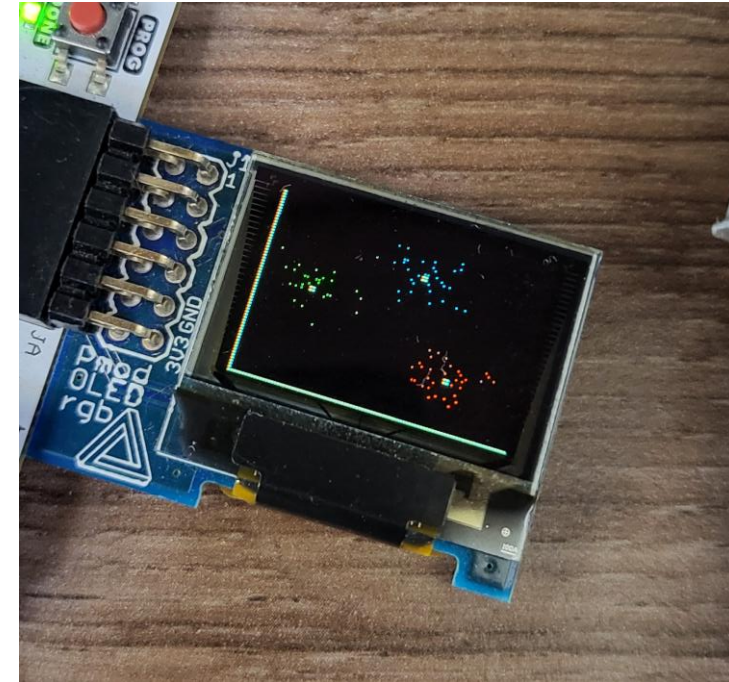
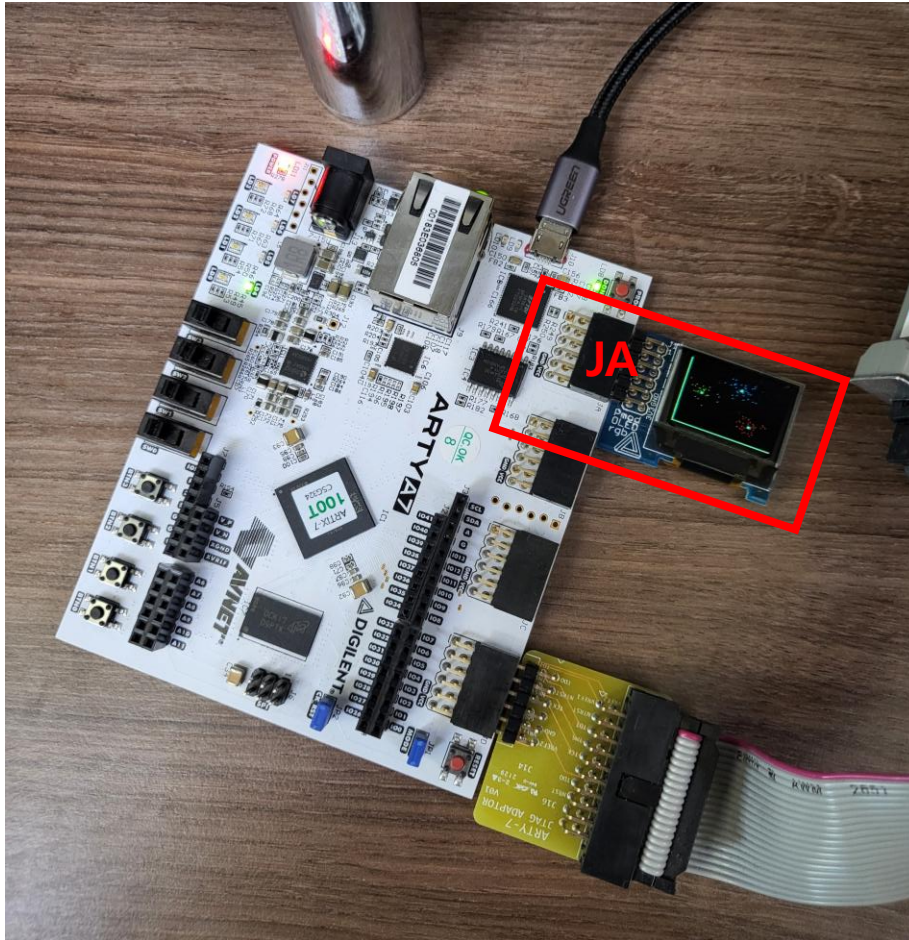
**full\_printf : 0 or 1** (sim\_rtl에서 printf수행속도가 매우 느리기 때문에 printf를 최소화하기 위해 만들어둔 옵션.)

**RTL : 0 권장**  
**FPGA : 0 or 1**

# Questa에서 Radix 설정 방법



# Pmod OLED RGB 연결 방법

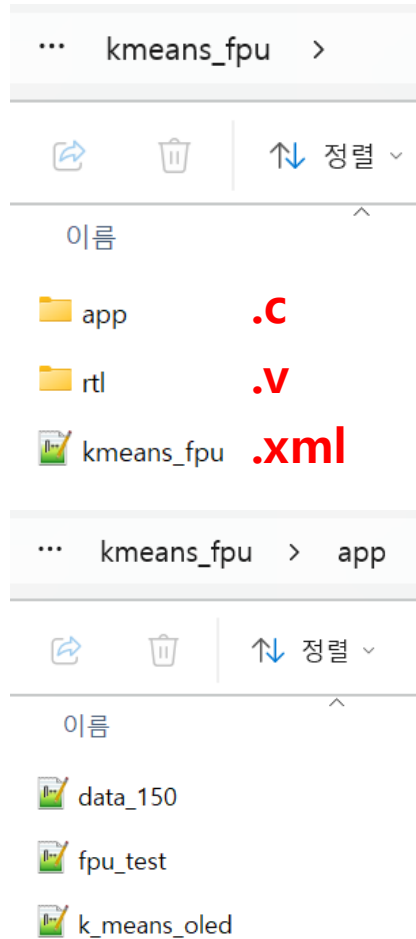


※ K-means clustering (data 100)  
실행 완료 시 OLED 출력화면

## [Contents]

01. K-means clustering
02. FPU (Floating Point Unit)
03. 프로젝트 안내
- 04. 첨부 파일 (kmeans\_fpu.zip)**

# 첨부 파일 (kmeans\_fpu.zip)



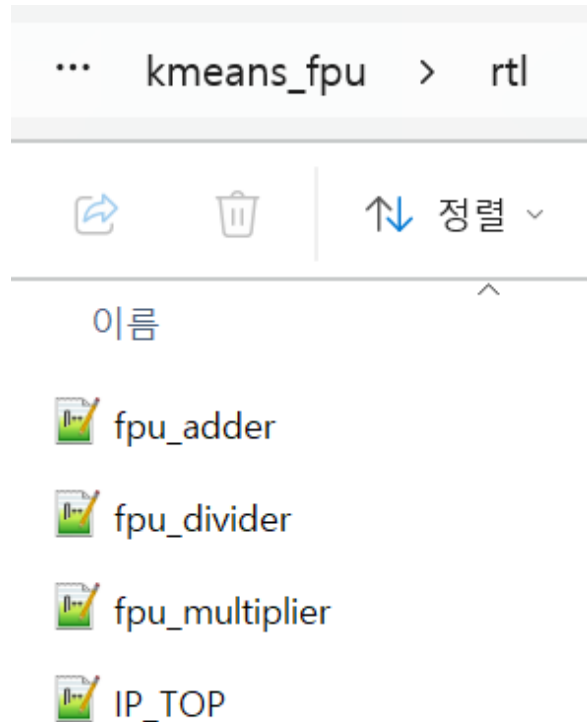
✓ **kmeans\_fpu.xml** : 플랫폼 xml 파일(수정 X, 그대로 사용.)

✓ **fpu\_test.c** : Step2 테스트용 app.

✓ **k\_means\_oled.c** : K-means clustering 어플리케이션

✓ **data\_150.h** : K-means clustering 어플리케이션에 사용되는 데이터

# 첨부 파일 (kmeans\_fpu.zip)



✓ **fpu\_adder, fpu\_divider, fpu\_multiplier** : FPU 모듈 verilog 코드 (수정 X)

✓ **IP\_TOP** : IP top 모듈.

(주석처리 된 빈 칸에 코드를 작성해주시길 바랍니다.)

(그 외의 부분은 수정 X)

감사합니다